

METHODS IN JAVA

An
Introduction

OBJECTIVES

- Define method, parameters and arguments.
- Discuss how methods are used in writing programs in Java.
- Write a Java program implementing methods.
- Compare programs written in Java using methods and without methods.

OPENING

PROBLEM

Find the sum of integers from 1 to 10, from 20 to 30, and from 35 to 45, respectively.

PROBLE

```
M
int sum = 0;
for (int i = 1; i <= 10; i++)
    sum += i;
System.out.println("Sum from 1 to 10 is " + sum);
```

```
sum = 0;
for (int i = 20; i <= 30; i++)
    sum += i;
System.out.println("Sum from
20 to 30 is " + sum);
```

```
sum = 0;
for (int i = 35; i <= 45; i++)
    sum += i;
System.out.println("Sum from
35 to 45 is " + sum);
```

PROBLE

M

```
int sum = 0;  
for (int i = 1; i <= 10; i++)  
    sum += i;  
System.out.println("Sum from 1 to 10 is " + sum);
```

```
sum = 0;  
for (int i = 20; i <= 30; i++)  
    sum += i;  
System.out.println("Sum from 20 to 30 is " + sum);
```

```
sum = 0;  
for (int i = 35; i <= 45; i++)  
    sum += i;  
System.out.println("Sum from  
35 to 45 is " + sum);
```

SOLUTION

N

```
public static int sum(int i1, int i2) {  
    int sum = 0;  
    for (int i = i1; i <= i2; i++)  
        sum += i;  
    return sum;  
}
```

```
public static void main(String[] args)  
{ System.out.println("Sum from 1 to 10 is " + sum(1, 10));  
  System.out.println("Sum from 20 to 30 is " + sum(20, 30));  
  System.out.println("Sum from 35 to 45 is " + sum(35, 45));  
}
```

WHAT IS A METHOD?

- A **method** is a block of code which only runs when it is called.
 - A method is a collection of statements that are grouped together to perform an operation.
 - You can pass data, known as parameters, into a method.
 - Methods are used to perform certain actions, and they are also known as **functions**.
 - Why use methods? To reuse code: define the code once and use it many times.
-

CREATE A METHOD

- A method must be declared within a class. It is defined with the name of the method, followed by parentheses ().
- Java provides some pre-defined methods, such as `System.out.println()`, but you can also create your own methods to perform certain actions:

Example: Create a method inside Main:

```
public class Main {  
    static void myMethod() {  
        // code to be executed  
    }  
}
```

EXAMPLE EXPLAINED

- myMethod() is the name of the method
- static means that the method belongs to the Main class and not an object of the Main class.
- void means that this method does not have a return value.

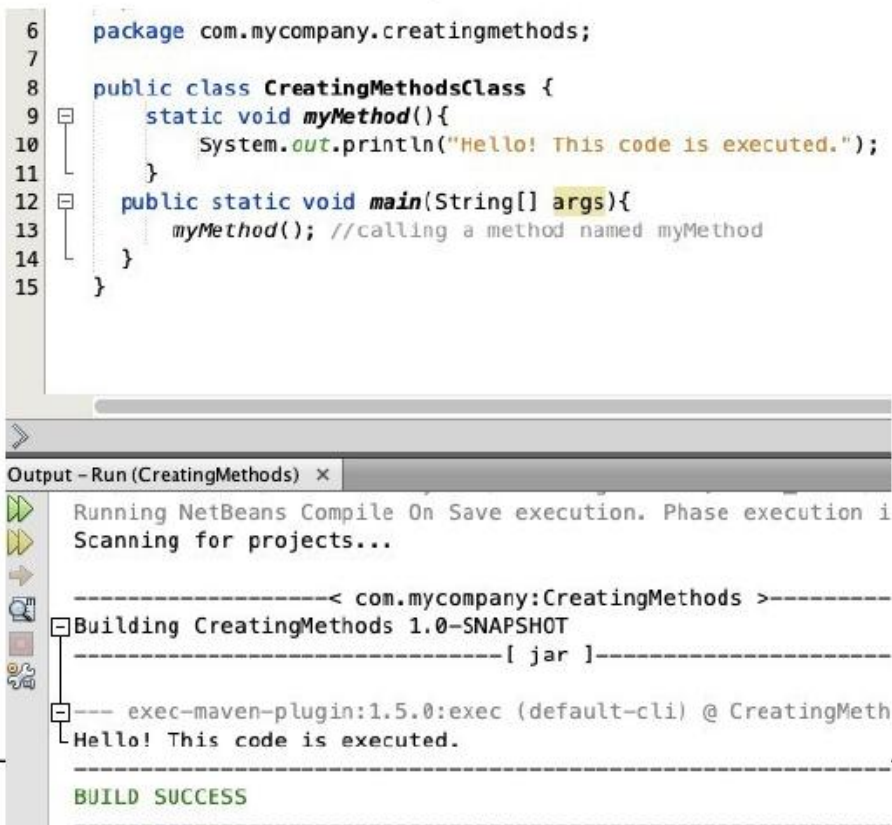
```
public class Main {  
    · static void myMethod() {  
        // code to be executed  
    }  
}
```

CALL A METHOD

- To call a method in Java, write the method's name followed by two parentheses () and a semicolon;
- In the following example, myMethod() is used to print a text (the action), when it is called:

Inside main, call the myMethod() method:

```
6 package com.mycompany.creatingmethods;
7
8 public class CreatingMethodsClass {
9     static void myMethod(){
10         System.out.println("Hello! This code is executed.");
11     }
12     public static void main(String[] args){
13         myMethod(); //calling a method named myMethod
14     }
15 }
```



Output - Run (CreatingMethods) x

Running NetBeans Compile On Save execution. Phase execution i
Scanning for projects...

-----< com.mycompany:CreatingMethods >-----
-Building CreatingMethods 1.0-SNAPSHOT
-----[jar]-----
-exec-maven-plugin:1.5.0:exec (default-cli) @ CreatingMeth
Hello! This code is executed.

BUILD SUCCESS

A METHOD
CAN ALSO
BE CALLED
MULTIPLE
TIMES:

```
6 package com.mycompany.creatingmethods;
7
8 public class CreatingMethodsClass {
9     static void myMethod(){
10         System.out.println("Hello! This code is executed.");
11     }
12     public static void main(String[] args){
13         myMethod(); //calling a method named myMethod
14         myMethod();
15         myMethod();
16     }
17 }
```

Output - Run (CreatingMethods) x

```
< com.mycompany:CreatingMethods >-----
Building CreatingMethods 1.0--SNAPSHOT
-----[ jar ]-----
--- exec-maven-plugin:1.5.0:exec (default-cli) @ CreatingMeth
Hello! This code is executed.
Hello! This code is executed.
Hello! This code is executed.
-----
BUILD SUCCESS
-----
```

PARAMETERS AND ARGUMENTS

- Information can be passed to methods as parameter.
 - Parameters act as variables inside the method.
 - *Parameters are specified after the method name, inside the parentheses. You can add as many parameters as you want, just separate them with a comma.*
-

PARAMETERS AND ARGUMENTS

- The following example has a method that takes a String called **firstname** as parameter.
- When the method is called, we pass along a first name, which is used inside the method to print the full name:

```
6 package com.mycompany.creatingmethods;
7
8 public class CreatingMethodsClass {
9     static void myMethod(String firstname){
10         System.out.println(firstname + " Medina");
11     }
12     public static void main(String[] args){
13         myMethod("Angela");
14         myMethod("Mark");
15         myMethod("Cristelle");
16     }
17 }
```

Output - Run (CreatingMethods) x

-----< com.mycompany:CreatingMethods >
Building CreatingMethods 1.0-SNAPSHOT
-----[jar]-----
----- exec-maven-plugin:1.5.0:exec (default-cli) @ Cre
Angela Medina
Mark Medina
Cristelle Medina

BUILD SUCCESS

PARAMETERS AND ARGUMENTS

- When a **parameter** is passed to the method, it is called an **argument**. So, from the example: `firstname` is a **parameter**, while `Angela`, `Mark` and `Cristelle` are **arguments**.

```
6 package com.mycompany.creatingmethods;
7
8 public class CreatingMethodsClass {
9     static void myMethod(String firstname){
10         System.out.println(firstname + " Medina");
11     }
12     public static void main(String[] args){
13         myMethod("Angela");
14         myMethod("Mark");
15         myMethod("Cristelle");
16     }
17 }
```

Output - Run (CreatingMethods) x

```
-----< com.mycompany:CreatingMethods >
Building CreatingMethods 1.0-SNAPSHOT
-----[ jar ]-----
exec-maven-plugin:1.5.0:exec (default-cli) @ Cre
Angela Medina
Mark Medina
Cristelle Medina
BUILD SUCCESS
```

MULTIPLE PARAMETER S

- You can have as many parameters as you like:

```
6 package com.mycompany.creatingmethods;
7
8 public class CreatingMethodsClass {
9     static void myMethod(String firstname, int age){
10         System.out.println(firstname + " Medina " + age);
11     }
12     public static void main(String[] args){
13         myMethod("Angela",18);
14         myMethod("Mark", 21);
15         myMethod("Cristelle", 33);
16     }
17 }
```

Output - Run (CreatingMethods) x

```
< com.mycompany:CreatingMethods >-----
Building CreatingMethods 1.0-SNAPSHOT
-----[ jar ]-----
exec-maven-plugin:1.5.0:exec (default-cli) @ CreatingM
Angela Medina 18
Mark Medina 21
Cristelle Medina 33
BUILD SUCCESS
```


RETURN VALUES

- The void keyword, used in the examples above, indicates that the method should not return a value.
- If you want the method to return a value, you can use a primitive data type (such as int, char, etc.) instead of void, and use the return keyword inside the method:

```
6 package com.mycompany.creatingmethods;  
7  
8 public class CreatingMethodsUsingReturn {  
9     static int myMethod2(int x){  
10         return 5 + x;  
11     }  
12  
13     public static void main (String[] args){  
14         System.out.println(myMethod2(3));  
15     }  
16 }  
17
```

Output - Run (CreatingMethodsUsingReturn) x

Running NetBeans Compile On Save execution. Phase 1
Scanning for projects...

-----< com.mycompany:CreatingMethods
Building CreatingMethods 1.0-SNAPSHOT
-----[jar]-----

----- exec-maven-plugin:1.5.0:exec (default-cli) @ C
8

BUILD SUCCESS


```

6 package com.mycompany.creatingmethods;
7
8 public class CreatingMethodsClass {
9     static void myMethod(String firstname, int age){
10         System.out.println(firstname + " Medina " + age);
11     }
12     public static void main(String[] args){
13         myMethod("Angela",18);
14         myMethod("Mark", 21);
15         myMethod("Cristelle", 33);
16     }
17 }

```

Output - Run (CreatingMethods) x

```

-----< com.mycompany:CreatingMethods >-----
Building CreatingMethods 1.0-SNAPSHOT
-----[ jar ]-----
exec-maven-plugin:1.5.0:exec (default-cli) @ CreatingM
Angela Medina 18
Mark Medina 21
Cristelle Medina 33
BUILD SUCCESS

```

```

6 package com.mycompany.creatingmethods;
7
8 public class CreatingMethodsUsingReturn {
9     static int myMethod2(int x){
10         return 5 + x;
11     }
12     public static void main (String[] args){
13         System.out.println(myMethod2(3));
14     }
15 }
16
17

```

Output - Run (CreatingMethodsUsingReturn) x

```

Running NetBeans Compile On Save execution. Phase (
Scanning for projects...

-----< com.mycompany:CreatingMethods
Building CreatingMethods 1.0-SNAPSHOT
-----[ jar ]-----
exec-maven-plugin:1.5.0:exec (default-cli) @ C
8
BUILD SUCCESS

```

THIS EXAMPLE RETURNS THE SUM OF A METHOD'S TWO PARAMETERS

:

```
6 package com.mycompany.creatingmethods;  
7  
8 public class CreatingMethodsUsingReturn {  
9     static int myMethod2(int x, int y){  
10         return x + y;  
11     }  
12  
13     public static void main (String[] args){  
14         System.out.println(myMethod2(3,15));  
15     }  
16 }  
17
```

Output - Run (CreatingMethodsUsingReturn) x

Running NetBeans Compile On Save execution. Phas Scanning for projects...

-----< com.mycompany:CreatingMetho

Building CreatingMethods 1.0-SNAPSHOT

-----[jar]-----

---- exec-maven-plugin:1.5.0:exec (default-cli) @

18

BUILD SUCCESS

- You can also store the result in a variable (recommended, as it is easier to read and maintain):

```
6 package com.mycompany.creatingmethods;
7
8 public class CreatingMethodsUsingReturn {
9     static int myMethod2(int x, int y){
10         return x + y;
11     }
12
13     public static void main (String[] args){
14         int z= myMethod2(3, 15);
15         System.out.println(z);
16     }
17 }
18
```

com.mycompany.creatingmethods.CreatingMethodsUsingReturn

Output - Run (CreatingMethodsUsingReturn) x

Running NetBeans Compile On Save execution. Phase Scanning for projects...

-----< com.mycompany:CreatingMetho

Building CreatingMethods 1.0-SNAPSHOT

-----[jar]-----

exec-maven-plugin:1.5.0:exec (default-cli) @

18

BUILD SUCCESS

A METHOD WITH IF...ELSE

- It is common to use if...else statements inside methods:

```
6 package com.mycompany.creatingmethods;
7
8 public class MethodWithIfElse {
9     static void EligibleToVote(int age){
10         if (age>=18){
11             System.out.println("You are eligible to vote!");
12         } else{
13             System.out.println("You are not eligible to vote!");
14         }
15     }
16 }
17
18 public static void main(String[] args){
19     EligibleToVote(28);
20 }
21 }
```

Output - Run (MethodWithIfElse) x

Running NetBeans Compile On Save execution. Phase execution is ski
Scanning for projects...

-----< com.mycompany:CreatingMethods >-----
Building CreatingMethods 1.0-SNAPSHOT
-----[jar]-----

--- exec-maven-plugin:1.5.0:exec (default-cli) @ CreatingMethods ---
You are eligible to vote!

BUILD SUCCESS

JAVA METHOD OVERLOADING

- With **method overloading**, multiple methods can have the same name with different parameters:

Example:

```
int myMethod(int x)
```

```
float myMethod(float x)
```

```
double myMethod(double  
x, double y)
```

JAVA METHOD OVERLOADING

- Consider the following example, which has two methods that add numbers of different type:

```
6 package com.mycompany.creatingmethods;
7
8 public class MethodOverloadingSample {
9     static int MethodInt(int x, int y){
10         return x+y;
11     }
12     static double MethodDouble(double x, double y){
13         return x+y;
14     }
15     public static void main (String[] args){
16         int num1=MethodInt(3,15);
17         double num2=MethodDouble(5.25,3.75);
18
19         System.out.println("Integer: "+ num1);
20         System.out.println("Double: "+ num2);
21     }
22 }
```

Output - Run (MethodOverloadingSample) x



Scanning for projects...



-----< com.mycompany:CreatingMethods >-----

Building CreatingMethods 1.0-SNAPSHOT

-----[jar]-----

----- exec-maven-plugin:1.5.0:exec (default-cli) @ Creati

Integer: 18

Double: 9.0

JAVA METHOD OVERLOADING

- Instead of defining two methods that should do the same thing, it is better to overload one.
- In the example below, we overload the AdditionMethod method to work for both int and double:

```
8 public class MethodOverloadingSample2 {
9     static int AdditionMethod(int x, int y){
10         return x+y;
11     }
12
13     static double AdditionMethod(double x, double y){
14         return x+y;
15     }
16
17     public static void main(String[] args){
18         int num1=AdditionMethod(15,3);
19         double num2=AdditionMethod(3.5, 5.75);
20
21         System.out.println("Sum int: "+num1);
22         System.out.println("Sum double: "+num2);
23     }
24 }
```

com.mycompany.creatingmethods.MethodOverloadingSample2

Output - Run (MethodOverloadingSample2) x

Scanning for projects...

-----< com.mycompany:CreatingMethods >-----

[-] Building CreatingMethods 1.0-SNAPSHOT

-----[jar]-----

[-] --- exec-maven-plugin:1.5.0:exec (default-cli) @ Creati

Sum int: 18

Sum double: 9.25

BUILD SUCCESS

```

6 package com.mycompany.creatingmethods;
7
8 public class MethodOverloadingSample {
9     static int MethodInt(int x, int y){
10         return x+y;
11     }
12     static double MethodDouble(double x, double y){
13         return x+y;
14     }
15     public static void main (String[] args){
16         int num1=MethodInt(3,15);
17         double num2=MethodDouble(5.25,3.75);
18
19         System.out.println("Integer: "+ num1);
20         System.out.println("Double: "+ num2);
21     }
22 }

```

Output - Run (MethodOverloadingSample) x

```

Scanning for projects...

-----< com.mycompany:CreatingMethods >-----
[Building CreatingMethods 1.0-SNAPSHOT]
[ jar ]-----

exec-maven-plugin:1.5.0:exec (default-cli) @ Creati
Integer: 18
Double: 9.0

```

```

8 public class MethodOverloadingSample2 {
9     static int AdditionMethod(int x, int y){
10         return x+y;
11     }
12
13     static double AdditionMethod(double x, double y){
14         return x+y;
15     }
16
17     public static void main(String[] args){
18         int num1=AdditionMethod(15,3);
19         double num2=AdditionMethod(3.5, 5.75);
20
21         System.out.println("Sum int: "+num1);
22         System.out.println("Sum double: "+num2);
23     }
24 }

```

com.mycompany.creatingmethods.MethodOverloadingSample2 x

Output - Run (MethodOverloadingSample2) x

```

Scanning for projects...

-----< com.mycompany:CreatingMethods >-----
[Building CreatingMethods 1.0-SNAPSHOT]
[ jar ]-----

exec-maven-plugin:1.5.0:exec (default-cli) @ Creati
Sum int: 18
Sum double: 9.25

BUILD SUCCESS

```

JAVA SCOPE

- In Java, variables are only accessible inside the region they are created.
- *This is called **scope**.*

METHOD SCOPE

- *Variables declared directly inside a method are available anywhere in the method following the line of code in which they were declared:*

```
public class Main {  
    public static void main(String[] args) {  
  
        // Code here CANNOT use x  
  
        int x = 100;  
  
        // Code here can use x  
        System.out.println(x);  
    }  
}
```

BLOCK SCOPE

- A block of code refers to all of the code between curly braces {}.
- Variables declared inside blocks of code are only accessible by the code between the curly braces, which follows the line in which the variable was declared:

```
public class Main {  
    public static void main(String[] args) {  
  
        // Code here CANNOT use x  
  
        { // This is a block  
  
            // Code here CANNOT use x  
  
            int x = 100;  
  
            // Code here CAN use x  
            System.out.println(x);  
  
        } // The block ends here  
  
        // Code here CANNOT use x  
  
    }  
}
```

BLOCK SCOPE

- A block of code may exist on its own or it can belong to an *if, while or for* statement.
 - In the case of *for* statements, variables declared in the statement itself are also available inside the block's scope.
-

SAMPLE PROGRA M USING METHOD S

```
6 package com.mycompany.creatingmethods;
7
8 public class ArithmeticMethods {
9     static int AdditionMethod(int x, int y){
10         return x+y;
11     }
12
13     static int SubtractionMethod(int x, int y){
14         return x-y;
15     }
16
17     public static void main (String[] args){
18
19         int num1=AdditionMethod(5,3);
20         int num2=SubtractionMethod(5,3);
21         System.out.println("The sum is: " + num1);
22         System.out.println("The difference is: " + num2);
23     }
24 }
```

Output - Run (ArithmeticMethods) x



Scanning for projects...

-----< com.mycompany:CreatingMethods >-----

Building CreatingMethods 1.0-SNAPSHOT

-----[jar]-----

--- exec-maven-plugin:1.5.0:exec (default-cli) @ CreatingMethods

The sum is: 8

The difference is: 2

BUILD SUCCESS

Total time: 0.941 s

Finished at: 2021-12-15T01:45:17+08:00

```
mirror_mod = modifier_ob.  
# Set mirror object to mirror  
mirror_mod.mirror_object =  
operation == "MIRROR_X":  
mirror_mod.use_x = True  
mirror_mod.use_y = False  
mirror_mod.use_z = False  
operation == "MIRROR_Y":  
mirror_mod.use_x = False  
mirror_mod.use_y = True  
mirror_mod.use_z = False  
operation == "MIRROR_Z":  
mirror_mod.use_x = False  
mirror_mod.use_y = False  
mirror_mod.use_z = True  
# Selection at the end - add  
mirror_ob.select = 1  
modifier_ob.select = 1  
context.scene.objects.active  
("Selected" + str(modifier_ob.name))  
mirror_ob.select = 0  
bpy.context.selected_objects  
data.objects[one.name].select  
print("please select exactly one mirror")  
----- OPERATOR CLASSES -----
```

```
types.Operator):  
X mirror to the selected  
object.mirror_mirror_x"  
mirror X"
```

```
context):  
context.active_object is not None
```

YOUR ASSIGNMENT:

1. Based on the given sample program, add a method to multiply two integers and a method to divide two integers.
2. Apply method overloading to minimize the given sample code.

ACCEPTING INPUT FROM THE USER

```
6 package com.mycompany.creatingmethods;
7 import java.util.Scanner;
8
9 public class ArithmeticAcceptInputMethod {
10     static int AdditionMethod(int x, int y){
11         return x+y;
12     }
13     static int SubtractionMethod(int x, int y){
14         return x-y;
15     }
16     public static void main (String[] args){
17         int num1;
18         int num2;
19         Scanner input=new Scanner (System.in);
20         System.out.println("Enter 1st number: ");
21         num1=input.nextInt();
22
23         System.out.println("Enter 2nd number: ");
24         num2=input.nextInt();
25
26         System.out.print("The sum is " );
27         System.out.println(AdditionMethod(num1,num2));
28         System.out.print("The difference is " );
29         System.out.println(SubtractionMethod(num1,num2));
30     }
31 }
```

```
-----< com.
Building CreatingMethods
-----
exec-maven-plugin:1.5
Enter 1st number:
30
Enter 2nd number:
15
The sum is 45
The difference is 15
-----
BUILD SUCCESS
-----
```

REFERENCES:

- [https://
www.w3schools.com/java/java_methods.asp](https://www.w3schools.com/java/java_methods.asp)